

DEPLOYMENT RUNBOOK

Dechub Backend Hosting Guide

A clear, repeatable guide for deploying the Node.js, Express, and MongoDB backend to Render, connecting MongoDB Atlas, and updating the Vercel frontend.

Prepared for Dechub Bridge • September 2026

Use placeholders for all credentials. Never place real secrets in source control or this document.

1. Architecture

```
Browser
| https://bridge.dechub.ai
v
Vercel frontend
| VITE_API_URL
v
Render Express API
| MONGODB_URI
v
MongoDB Atlas
```

The frontend is hosted on Vercel. The Express API is hosted as a Render Web Service. MongoDB Atlas hosts the database. The frontend calls the API using `VITE_API_URL`.

2. Before You Start

- A GitHub repository containing the `Server` folder.
- A MongoDB Atlas account and active cluster.
- A Render account connected to GitHub.
- A Vercel project serving `https://bridge.dechub.ai`.
- New, securely stored production credentials for MongoDB, JWT, Google, SMTP, and enabled integrations.

Security first: If a password, private key, URI, or API key was pasted into a chat, terminal capture, screenshot, or public repository, revoke/rotate it before launching production.

3. MongoDB Atlas

3.1 Create the cluster

1. Sign in at mongodb.com/atlas.
2. Create a project, for example `Dechub`.
3. Select **Create** and create an Atlas database deployment.
4. Choose a region close to the Render API region. Keeping Atlas and Render close reduces database latency.
5. Wait until the cluster status is active.

3.2 Create a database user

1. Open **Database & Network Access** and then **Database Users**.
2. Select **Add New Database User**.
3. Create an application user, for example `dechub_app`, with a long generated password.

4. Grant `readWrite` access to the `dechub` database, or use broader access temporarily while setting up.
5. Store the password in a password manager. Atlas does not reveal it again.

3.3 Allow the Render service to connect

1. Open **Network Access** in Atlas.
2. Click **Add IP Address**.
3. For the Render Free plan, choose **Allow Access from Anywhere**, which adds `0.0.0.0/0`.
4. Save and wait briefly for the rule to apply.

Production note: Render Free does not provide a fixed outbound IP. `0.0.0.0/0` is acceptable for initial deployment only when paired with a strong dedicated database user. For a hardened setup, use static egress IP/private networking and restrict the Atlas access list.

3.4 Copy the Atlas URI

1. Open the Atlas cluster and select **Connect**.
2. Select **Drivers**, then choose **Node.js**.
3. Copy the displayed `mongodb+srv://...` URI.
4. Replace the password placeholder with the database-user password. URL-encode special password characters if required.

```
MONGODB_URI=mongodb+srv://<database-user>:<encoded-password>@<cluster-host>/?  
retryWrites=true&w=majority
```

4. Prepare the Repository

1. Ensure `Server/package.json` and `Server/package-lock.json` are committed and synchronized.
2. Do not commit `Server/.env`.
3. Commit the current server lockfile before connecting Render:

```
git add Server/package-lock.json
git commit -m "Fix server lockfile for Render build"
git push origin main
```

Ensure the server build works locally:

```
cd Server
npm ci --include=dev
npm run build
```

5. Create the Render Web Service

1. Open [Render Dashboard](#) and select **New > Web Service**.
2. Connect the Dechub GitHub repository.
3. Configure the service as follows.

Setting	Value
Name	<code>dechub-bridge-api</code> (or another unique service name)
Language	Node
Branch	<code>main</code>
Root Directory	<code>Server</code>
Build Command	<code>npm ci --include=dev && npm run build</code>
Start Command	<code>npm start</code>
Health Check Path	<code>/health</code>

Add this Render environment variable to pin the Node version:

```
NODE_VERSION=20.11.0
```

Why include dev dependencies? TypeScript and `@types/*` packages are needed to compile the server, but are listed in `devDependencies`. `--include=dev` ensures the build can access them even though the runtime uses `NODE_ENV=production`.

6. Configure Render Environment Variables

In Render, open the service and select **Environment**. Add each name and value separately. Do not paste literal angle brackets from this guide.

Variable	Production value
<code>NODE_ENV</code>	<code>production</code>
<code>CLIENT_URL</code>	<code>https://bridge.dechub.ai</code> (no trailing slash)
<code>MONGODB_URI</code>	The new MongoDB Atlas URI
<code>JWT_SECRET</code>	A newly generated random secret, at least 32 characters
<code>JWT_REFRESH_SECRET</code>	A different newly generated random secret, at least 32 characters
<code>JWT_EXPIRES_IN</code>	<code>15m</code>
<code>JWT_REFRESH_EXPIRES_IN</code>	<code>7d</code>
<code>GOOGLE_CLIENT_ID</code>	Google OAuth client ID
<code>GOOGLE_CLIENT_SECRET</code>	Google OAuth client secret
<code>GOOGLE_CALLBACK_URL</code>	<code>https://<render-service>.onrender.com/api/auth/google/callback</code>
<code>PAYMENT_PROVIDER</code>	<code>dummy</code> , or <code>stripe</code> when Stripe is configured
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code>	Production SMTP provider settings
<code>EMAIL_FROM</code>	Approved sender address
<code>MAX_FILE_SIZE_MB</code>	<code>10</code>
<code>DECHUB_ADMIN_EMAILS</code>	Internal admin email address(es)
<code>DECHUB_ADMIN_PASSWORD</code>	A newly generated, unique admin password

Add Stripe, DocuSign, and Google Sheets variables only when those integrations are enabled.

Do not add: `PORT` (Render supplies it) or `DNS_SERVERS` (the local development workaround is not needed on Render).

File uploads: Free vs paid Render

Plan	Set <code>UPLOAD_DIR</code> to	Result
Free / testing	<code>uploads</code>	Files are ephemeral and can disappear after a restart or deploy.
Paid with a disk	<code>/var/data</code>	Attach a Render persistent disk mounted at <code>/var/data</code> .

7. Deploy and Verify the API

1. Select **Deploy Web Service**.
2. Wait for the build log to complete.
3. Open the health endpoint:

```
https://<render-service>.onrender.com/health
```

A healthy API returns JSON containing `"status": "ok"`. If the service reports no open port, check the Atlas connection and environment variables. The application does not begin listening until MongoDB connects.

Common Render errors

Symptom	Likely cause and action
<code>npm ci</code> says lockfile is not in sync	Regenerate and commit <code>Server/package-lock.json</code> , then redeploy.
Missing <code>@types/express</code> during build	Use <code>npm ci --include=dev && npm run build</code> .
MongoDB TLS / server-selection errors	Confirm the Atlas cluster is active, <code>MONGODB_URI</code> is correct, and Atlas Network Access permits the Render service.
CORS error in browser	Set <code>CLIENT_URL=https://bridge.dechub.ai</code> in Render, save, and redeploy.

8. Configure Google OAuth

1. Open Google Cloud Console and select the OAuth 2.0 Client ID used by Dechub.
2. Under **Authorized JavaScript origins**, add:

```
https://bridge.dechub.ai
```

3. Under **Authorized redirect URIs**, add:

```
https://<render-service>.onrender.com/api/auth/google/callback
```

The redirect URI must match the Render `GOOGLE_CALLBACK_URL` exactly.

9. Update the Vercel Frontend

1. Open the Vercel frontend project.
2. Go to **Settings > Environment Variables**.
3. Add a **Config** variable for Production:

```
VITE_API_URL=https://<render-service>.onrender.com
```

Do not add `/api` or a trailing slash. The frontend code already appends `/api`.

4. Save the variable.
5. Redeploy the latest Vercel production deployment. Vite variables are embedded at build time, so a redeploy is required.

10. Go-Live Checklist

- The API health endpoint returns `status: ok`.
- The Vercel production deployment uses the Render API URL.
- Registration, login, token refresh, logout, and password reset work.
- Google login works and returns to `bridge.dechub.ai`.
- SMTP email delivery is verified.
- Upload storage matches the chosen Render plan.
- All exposed credentials are rotated.
- All secrets exist only in Render/Vercel environment settings or a password manager.

11. Production Recommendations

- Use a paid Render plan for real users. The free plan has limited CPU and can make authentication noticeably slow.
- Place Render and Atlas in the same region. Place that region near your main user base when possible.
- Move KYC and contract documents from publicly served local uploads to private object storage with signed URLs before handling real customer data.
- Do not send access or refresh tokens in URL query strings. Use secure cookies or a short-lived one-time exchange code for OAuth.
- Use a dedicated database user with minimal privileges and tighten Atlas network access when static egress or private networking is available.